

33. Optimal Page Replacement

Aim

To write and execute a C program to simulate the **Optimal page replacement technique** of memory management and calculate the minimum possible number of page faults.

Algorithm

1. Start the program.
2. Read the number of pages in the reference string.
3. Read the page reference string.
4. Read the number of available frames.
5. Initialize all frames as empty.
6. Read each page from the reference string one by one.
7. Check whether the page is already present in any frame.
8. If the page is present, it is a **page hit** and no replacement is performed.
9. If the page is not present, it is a **page fault**.
10. If an empty frame is available, place the new page in it.
11. If all frames are occupied, look at the future page references.
12. Find the page whose **next use occurs farthest in the future**.
13. Replace that page with the new page.
14. If a page will not be used again, select that page for replacement.
15. Increment the page-fault count.
16. Display the contents of the frames after each page reference.
17. Repeat the process until all pages are processed.
18. Display the total number of page faults.
19. Stop the program.

Code:

```
#include <stdio.h>

int main() {
    int pages[50], frames[10];
    int n, f, i, j, k;
    int found, fault = 0;
    int pos, farthest, next;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string:\n");
    for (i = 0; i < n; i++)
        scanf("%d", &pages[i]);
```

```

printf("Enter number of frames: ");
scanf("%d", &f);
for (i = 0; i < f; i++)
    frames[i] = -1;
for (i = 0; i < n; i++) {
    found = 0;
    for (j = 0; j < f; j++) {
        if (frames[j] == pages[i]) {
            found = 1;
            break;
        }
    }
    if (!found) {
        fault++;
        pos = -1;
        for (j = 0; j < f; j++) {
            if (frames[j] == -1) {
                pos = j;
                break;
            }
        }
        if (pos == -1) {
            farthest = -1;
            pos = 0;
            for (j = 0; j < f; j++) {
                next = n + 1;
                for (k = i + 1; k < n; k++) {
                    if (frames[j] == pages[k]) {
                        next = k;
                        break;
                    }
                }
            }
        }
    }
}

```

```

        }
    }
    if (next > farthest) {
        farthest = next;
        pos = j;
    }
}

frames[pos] = pages[i];
}

printf("Page %d: ", pages[i]);
for (j = 0; j < f; j++)
    printf("%d ", frames[j]);

printf("\n");
}

printf("Total Page Faults = %d\n", fault);

return 0;
}

```

Output:

main.c	Run	Output
<pre> 1 #include <stdio.h> 2 3 int main() { 4 int pages[50], frames[10]; 5 int n, f, i, j, k; 6 int found, fault = 0; 7 int pos, farthest, next; 8 9 printf("Enter number of pages: "); 10 scanf("%d", &n); 11 12 printf("Enter page reference string:\n"); 13 for (i = 0; i < n; i++) 14 scanf("%d", &pages[i]); 15 16 printf("Enter number of frames: "); 17 scanf("%d", &f); 18 19 for (i = 0; i < f; i++) 20 frames[i] = -1; 21 22 for (i = 0; i < n; i++) { 23 found = 0; 24 25 for (j = 0; j < f; j++) { </pre>		<pre> Enter number of pages: 12 Enter page reference string: 1 2 3 4 1 2 5 1 2 3 4 5 Enter number of frames: 3 Page 1: 1 -1 -1 Page 2: 1 2 -1 Page 3: 1 2 3 Page 4: 1 2 4 Page 1: 1 2 4 Page 2: 1 2 4 Page 5: 1 2 5 Page 1: 1 2 5 Page 2: 1 2 5 Page 3: 3 2 5 Page 4: 4 2 5 Page 5: 4 2 5 Total Page Faults = 7 === Code Execution Successful === </pre>